

TAKE-HOME TECHNICAL ASSIGNMENT

SENIOR FULL-STACK .NET MODERNIZATION DEVELOPER

1. PURPOSE OF THIS ASSIGNMENT

This take-home assignment is the **first phase** of our technical interview process.

The goal is to understand how you:

- Design and build maintainable applications
- Work with databases in a real-world, production-support mindset
- Apply business constraints such as **soft deletion**
- Integrate backend logic, data access, and (optionally) frontend visualization
- Use AI responsibly while retaining ownership of decisions

This is **not an exam**.

We are not looking for a perfect or production-ready application.

We are interested in your **thinking, structure, and decision-making**.

2. USE OF AI TOOLS

- AI tools (e.g., ChatGPT, Copilot) **are allowed**
- You must be able to **explain and justify all code and design choices**
- AI-generated content without understanding will be identified during follow-up discussions

AI is treated as a **tool**, not a substitute for engineering judgment.

3. SUBMISSION INSTRUCTIONS

Please submit **one ZIP file** containing:

- Complete source code
- SQLite database file (.db)
- SQL scripts used to create or modify schema, views, or reusable queries
- A README.md file explaining:
 - How to run the application
 - Your database design decisions
 - How soft deletion is implemented
 - Any trade-offs or assumptions made
 - What you would improve with more time

The application should **build and run locally** using your instructions.

4. BUSINESS CONTEXT & DOMAIN

The system manages **Providers** and their associated **Licenses**.

This reflects real internal applications supported and maintained by our organization.

Key Organizational Rule

 **Hard deletion is not allowed.**

All delete operations must be implemented using **soft deletion**.

No soft-delete columns or flags are provided upfront — you are expected to **design this behavior**.

5. REQUIRED DOMAIN CONCEPTS

Your solution must support the following concepts at minimum.

PROVIDER

- Unique identifier
- Provider name
- County
- Status or lifecycle state
- Soft-delete behavior
- Audit-safe persistence

LICENSE

- Unique identifier
- Association with a provider
- License number
- License status
- Expiration date

You may extend the model if needed.

6. DATA DICTIONARY (REFERENCE)

The following data dictionary describes the **business meaning** of the core entities.

You are responsible for designing the **physical SQLite schema** based on these definitions and the constraints of this assignment.

You may add additional fields or tables as necessary and should document them in your README.

ENTITY: PROVIDER

Represents an organization or individual that provides services and is managed by state staff.

- **ProviderId**
Unique identifier for the provider (primary key; implementation choice is yours)
- **ProviderName**
Display name of the provider; should be searchable
- **County**
County in which the provider operates (e.g., Fulton, DeKalb)
- **Status**
Business or lifecycle status of the provider
Example values: Active, Inactive, Pending
- **CreatedDate**
Date the provider record was created; used for audit and tracking

Notes

- Providers must not be physically deleted.
- Soft-deleted providers must not appear in standard application views.
- Soft-delete design decisions are a key evaluation area.

ENTITY: LICENSE

Represents a license issued to a provider that authorizes operations for a defined period.

- **LicenseId**
Unique identifier for the license (primary key)
- **ProviderId**
Reference to the associated provider (foreign key)
- **LicenseNumber**
Business-recognizable license identifier (usually unique per provider)
- **LicenseStatus**
Current status of the license
Example values: Active, Expired, Suspended
- **ExpirationDate**
Date on which the license expires; used to determine license validity

Notes

- A provider may have one or more licenses.
- Provider status does not necessarily imply license validity.

BUSINESS RULES & RELATIONSHIPS

- A Provider can have **zero or more Licenses**
- A License must belong to **one Provider**
- Providers may appear active while licenses are expired
- License expiration is determined by ExpirationDate
- Soft-deleted providers:
 - Must remain in the database
 - Must be excluded from standard views and operations
 - May still be queried for audit or troubleshooting

7. DATABASE REQUIREMENTS

- Use **SQLite** as the database
- You may use:
 - Tables
 - Views
 - Reusable query patterns
- Stored procedures are optional; SQLite-appropriate alternatives are acceptable

Soft Delete Expectations

Your design must ensure:

- Soft-deleted records do **not** appear in standard application results
- Deleted records remain queryable for audit or maintenance
- Soft-delete enforcement is consistent across queries
- The implementation is clearly explained in your README

8. APPLICATION FUNCTIONAL REQUIREMENTS

1. PROVIDER LISTING

- Display providers that are **not soft-deleted**
- Data source may be:

- Table queries
 - Views
 - Reusable data-access logic
- Explain your choice

2. CREATE PROVIDER

- Add new provider records
- Ensure compatibility with soft-delete rules

3. EDIT PROVIDER

- Update provider information
- Ensure data integrity

4. DELETE PROVIDER (SOFT DELETE ONLY)

- Implement deletion using your soft-delete design
- Deleted providers must not appear in normal application listings

5. REQUIRED DATA SCENARIOS

Your solution must support querying for:

- Active providers and their active licenses
- Providers appearing active but having expired licenses
- Exclusion of soft-deleted providers from standard results

These may be implemented using database objects, application logic, or a combination.

9. TECHNOLOGY STACK

- ASP.NET MVC (.NET 8/10)
- Razor Views
- Entity Framework Core (SQLite provider)
- Clean, readable code is valued over UI polish

10. OPTIONAL ENHANCEMENT – INTERACTIVE DASHBOARD (REACT)

THIS SECTION IS OPTIONAL.

Objective

Create a read-only dashboard highlighting provider and license insights.

Examples (choose any)

- Providers by status (bar or pie chart)
- Licenses per provider (bar chart)
- Expired vs active licenses (pie chart)
- Summary cards with key counts
- Providers with expiring licenses

Guidelines

- Use React (any setup)
- Charts may use any lightweight library

- Data must come from your backend (not hard-coded)
- Business logic must not exist only in the UI

If implemented, explain:

- Why you chose the metrics
- How frontend and backend are integrated

11. SOLUTION STRUCTURE

You are free to structure the solution based on your experience and best practices.

No specific folder structure is required.

Be prepared to explain your organizational choices.

12. WHAT WE EVALUATE

We evaluate submissions based on:

- Application architecture and clarity
- Database design and reasoning
- Quality of soft-delete implementation
- Backend and database integration
- Maintainability and production-support awareness
- Documentation quality
- Optional frontend depth (if implemented)

UI polish and advanced styling are **not primary criteria**.

13. WHAT HAPPENS NEXT

1. Submission review
2. Technical discussion / Q&A
3. On-site, hands-on interview for finalists
4. Final hiring decision

14. FINAL NOTES

- Partial completion is acceptable
- Clear reasoning is more important than completeness
- This assignment reflects real-world maintenance work

Thank you for your time and effort.

☒ SUBMISSION DEADLINE

Please submit your ZIP file as soon as possible. If a submission deadline is provided separately, please ensure your submission meets that deadline.